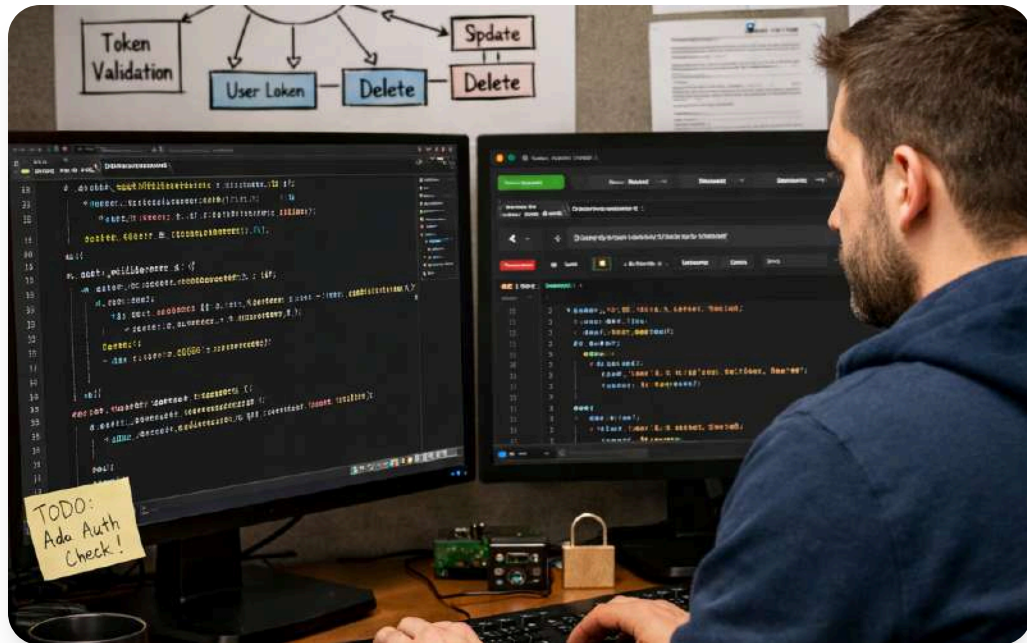




Golang Backend Pertemuan 5

Membahas collection, map, interface, dan abstraction di Go untuk backend yang rapi, modular, fleksibel, dan mudah dikembangkan.

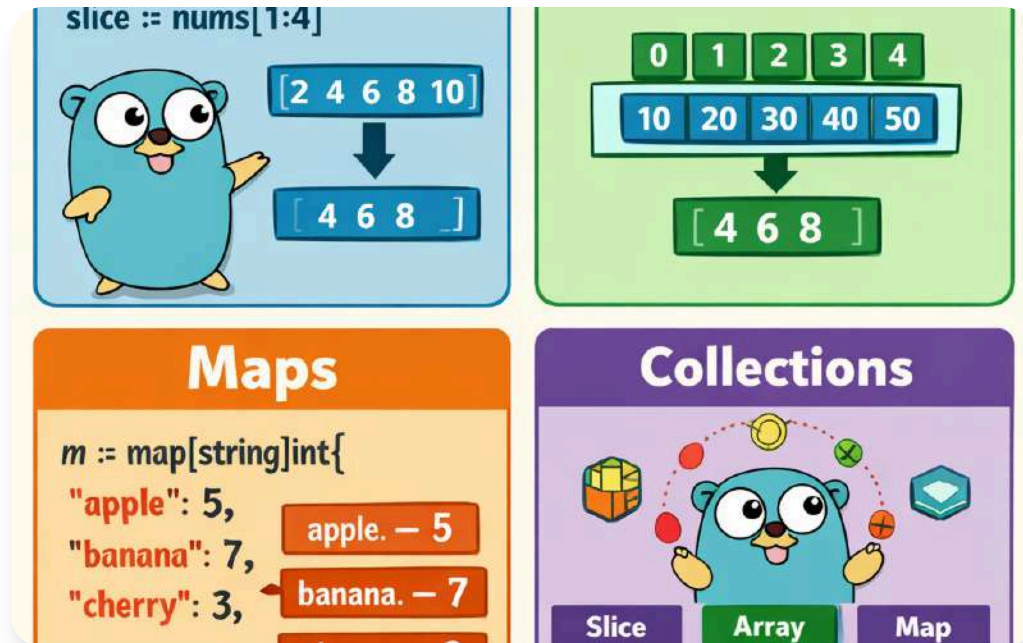


Juara Coding
June 2026



Tujuan Pembelajaran

Peserta mampu menjelaskan collection dasar Go, memakai map efektif, memahami interface, dan menerapkan abstraction backend sederhana.

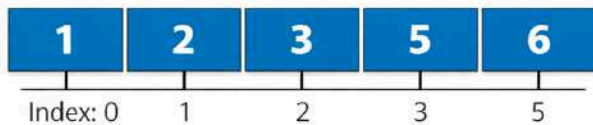


Collection di Go

Collection menyimpan banyak nilai dalam satu variabel. Di Go, bentuk umum meliputi **array**, **slice**, dan **map** untuk backend yang efisien.

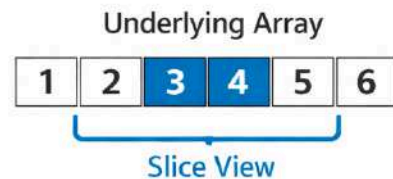
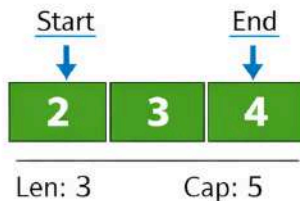
Array dan Slice

```
var arr [6]int = [1, 2, 3, 4, 5, 6]
```



Go Slice

```
slice == arr[1:4]
```



Array berukuran tetap, sedangkan *slice* lebih fleksibel untuk data dinamis pada backend modern.



Array Tetap

Array memiliki panjang tetap sehingga kurang fleksibel untuk kebutuhan data yang berubah.



Slice Fleksibel

Slice dapat bertambah atau berkurang ukuran sesuai kebutuhan proses aplikasi backend.



Umum di Backend

Slice sering dipakai untuk menampung hasil query, request, dan response.

Karakteristik Slice



Menyimpan Data Sejenis

Slice menampung kumpulan elemen dengan tipe yang sama untuk kebutuhan backend.



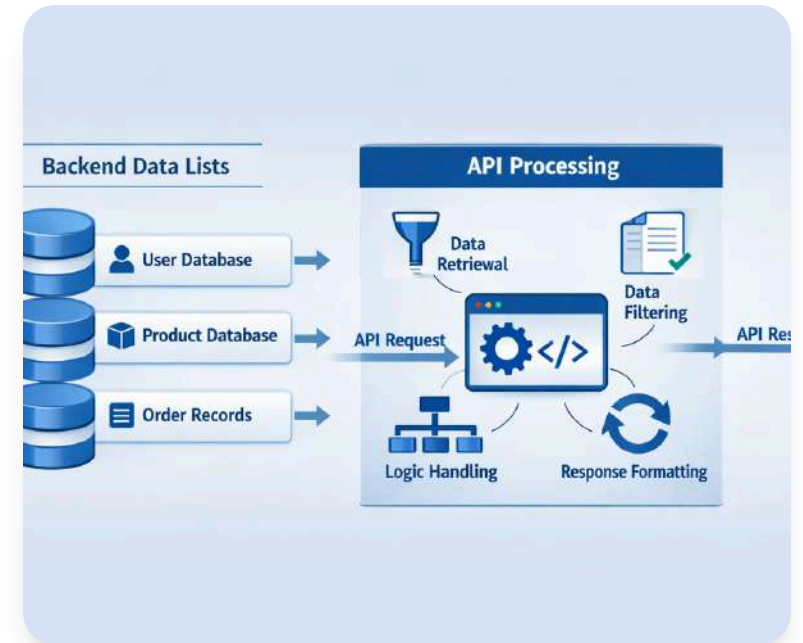
Operasi Fleksibel

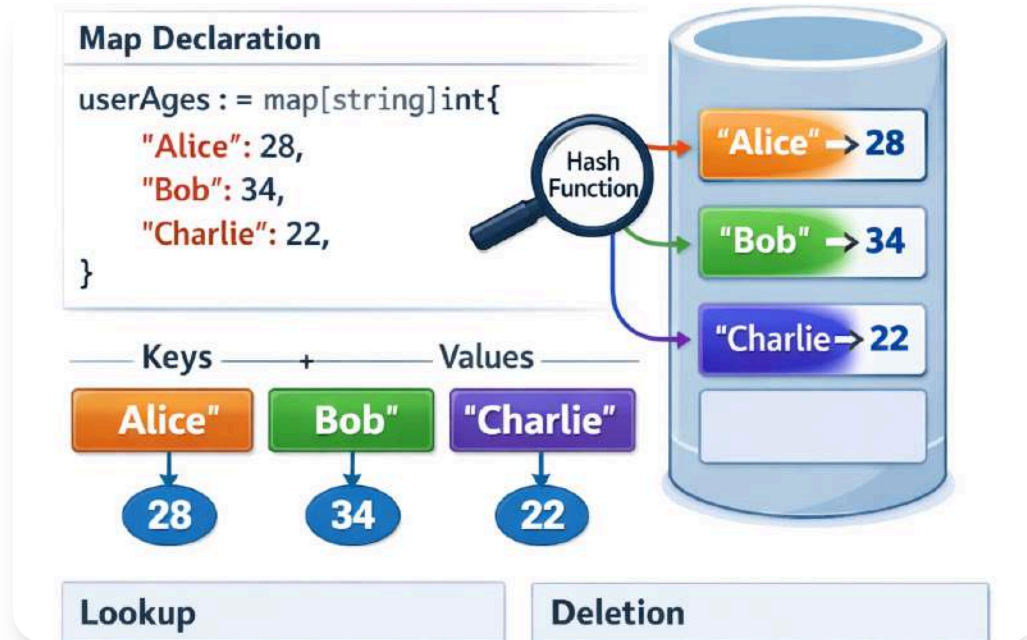
Mendukung append, slicing, dan iterasi untuk mengelola data secara efisien.



Umum di Backend

Sering dipakai untuk user, produk, transaksi, serta hasil data dari database dan API.

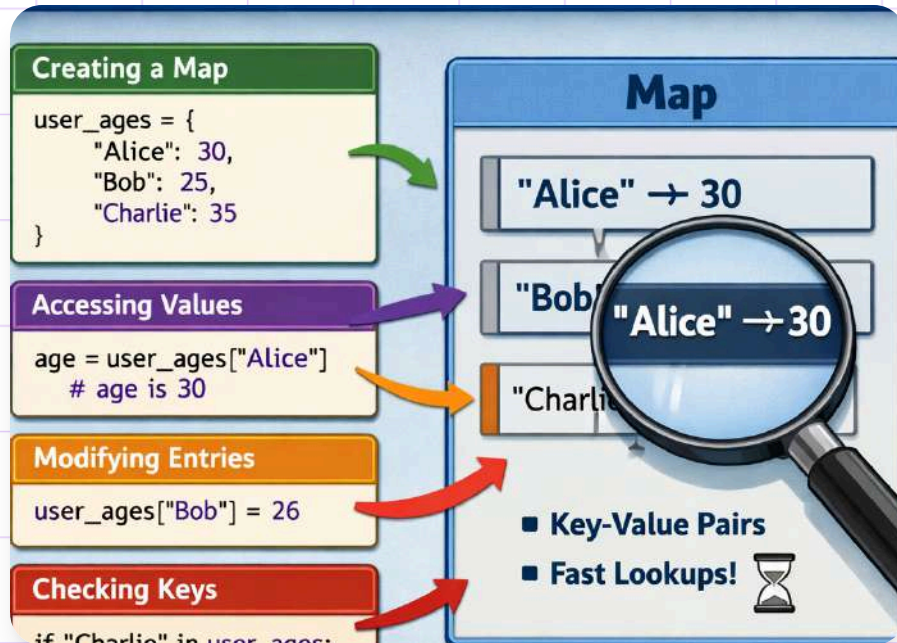




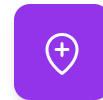
Pengenalan Map di Go

Map menyimpan pasangan key-value dan memudahkan akses data cepat berdasarkan kunci, seperti id, username, kode produk, atau konfigurasi backend.

Membuat dan Menggunakan Map



Map dibuat dengan make atau literal untuk menambah, mengubah, dan menghapus data secara lebih efisien.



Pembuatan Map

Map dapat dibuat menggunakan make atau dengan penulisan literal sesuai kebutuhan.



Kelola Data

Data dalam map bisa ditambahkan, diubah, dan dihapus dengan proses yang sederhana.



Lookup Efisien

Map memudahkan pencarian data tanpa perlu menelusuri slice berulang kali.


```
4
5 type User struct {
6     Name string
7     Age  int
8 }
9
10 func main() {
11     users := map[string]User{
12         "user1": {Name: "Alice", Age: 28},
13         "user2": {Name: "Bob", Age: 34},
14         "user3": {Name: "Charlie", Age: 22},
15     }
```

Contoh Map di Golang

Map **string ke string** menyimpan relasi ID user dan nama user. Contoh ini menambahkan data baru dan menghapus salah satu user secara sederhana.


```
f "go_map" in data:  
    go_map_value = data["go_map"]  
    print("go_map value:", go_map_value)  
else:  
    print("Key \"go_map\" is None or missing!")
```

Pemeriksaan Key pada Map

Di Go, akses map menghasilkan value dan penanda keberadaan key untuk memastikan data benar-benar ada sebelum diproses dan mengurangi risiko bug.

Iterasi pada Collection dan Map

```
fmt.Println("Index:", idx, "Value:", num)
}
```

```
// Iterating over a map
colors := map[string]string{
    "red": "#FF0000",
    "blue": "#0000FF",
}

for key, value := range colors {
    fmt.Println("Key:", key, "Value:", value)
}
```

01 Keyword range

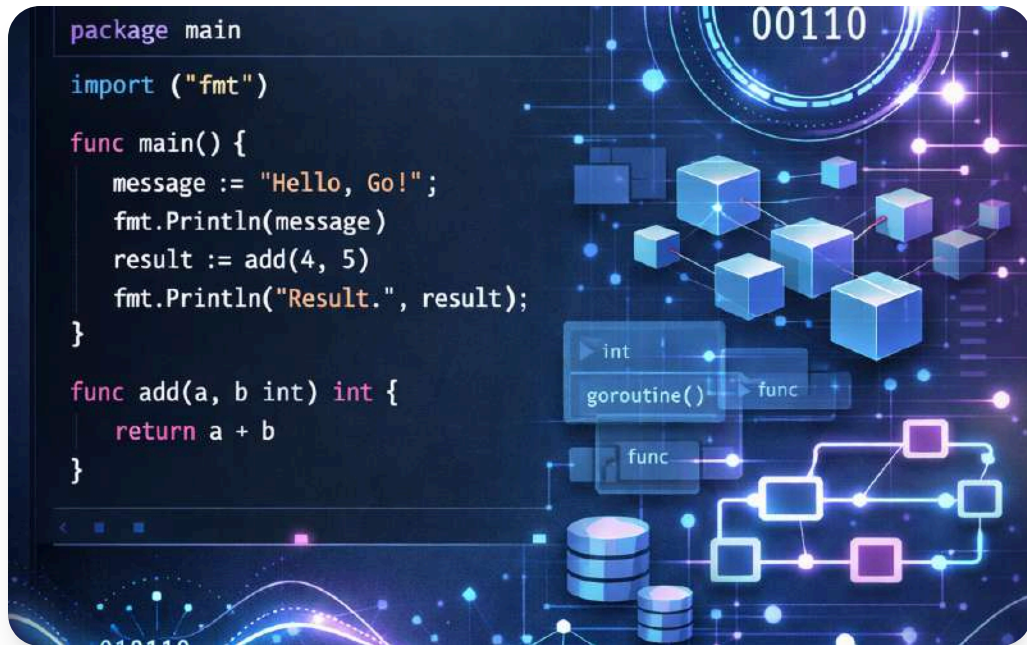
Go memakai keyword *range* untuk iterasi pada array, slice, dan map secara ringkas dan konsisten.

02 Pemrosesan daftar data

Iterasi sering digunakan untuk memproses daftar data, menyusun response, dan menjalankan validasi massal.

03 Transformasi collection

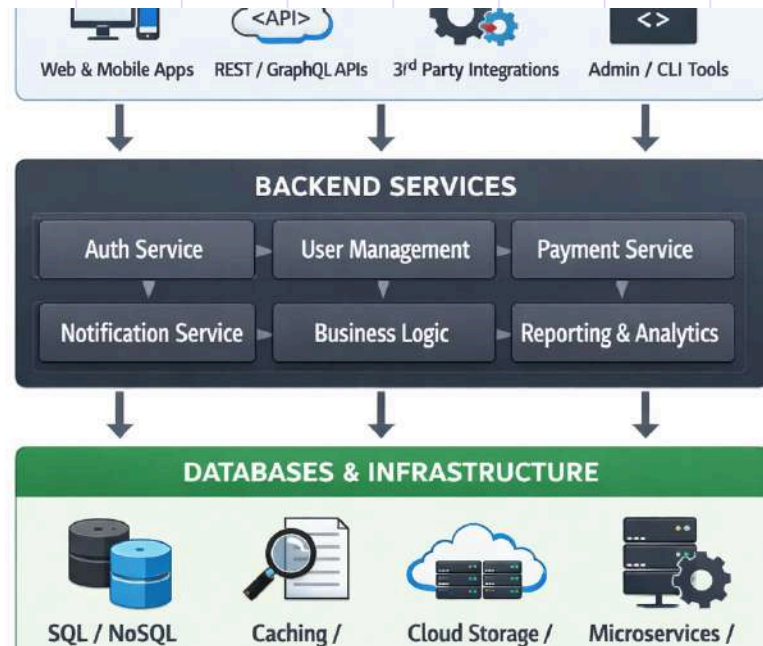
Isi collection dapat diubah terlebih dahulu sebelum diteruskan ke layer lain dalam aplikasi.



Interface dalam Go

Interface adalah kumpulan method signature yang mendefinisikan perilaku tipe, sehingga kode lebih fleksibel untuk berbagai implementasi.

Manfaat Interface Backend

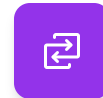


Interface memisahkan kontrak dari implementasi agar perubahan pada service, repository, dan adapter lebih mudah dikelola.



Kontrak Terpisah

Interface menjaga batas jelas antara definisi perilaku dan implementasi teknis.



Mudah Diganti

Database atau layanan eksternal dapat diganti dengan dampak perubahan yang lebih kecil.



Mendukung Testing

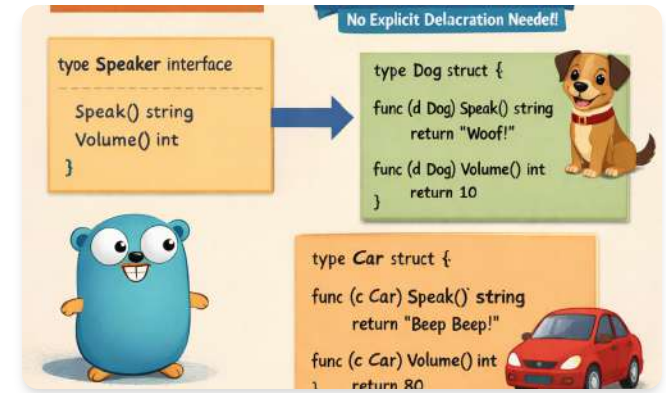
Mock testing lebih mudah karena dependensi dapat diganti melalui interface.

```
2  
3 type Shape_interface {  
4     Area() float64  
5     Perimeter() float64  
6 }
```

Contoh Interface Sederhana

Interface ini mendefinisikan kontrak untuk mengambil data user berdasarkan id, dengan implementasi dari memori, database, atau layanan eksternal.

Implementasi Interface



01 Implementasi Implisit

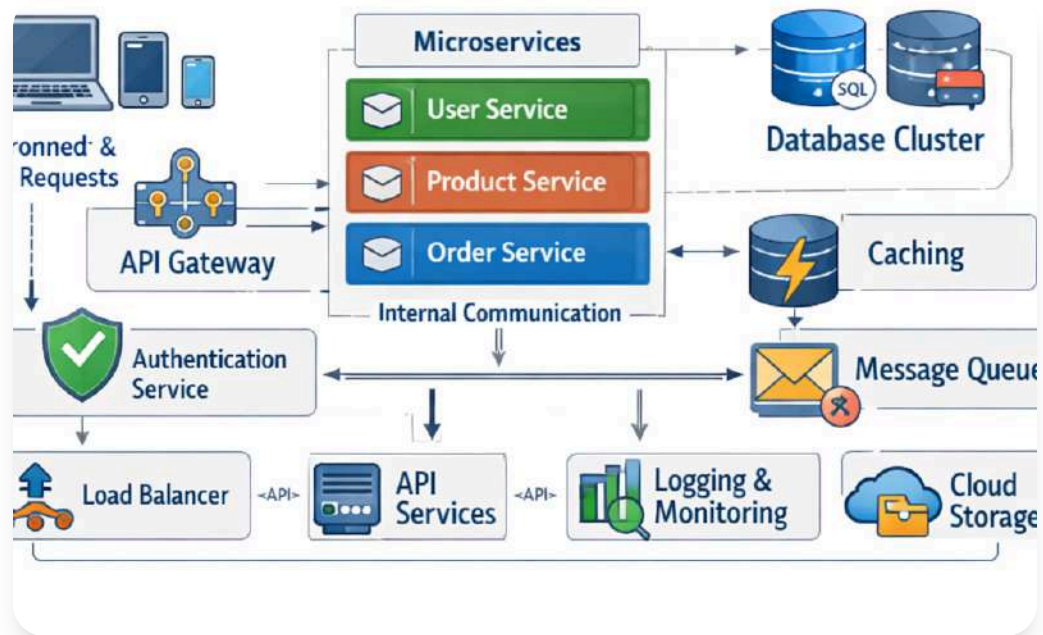
Sebuah struct memenuhi interface jika memiliki method yang sesuai, tanpa kata kunci khusus seperti implements.

02 Sederhana namun Kuat

Pendekatan ini membuat Go terasa ringkas, tetapi tetap kuat dalam membangun kontrak antar tipe.

03 Kontrak Tetap Terjaga

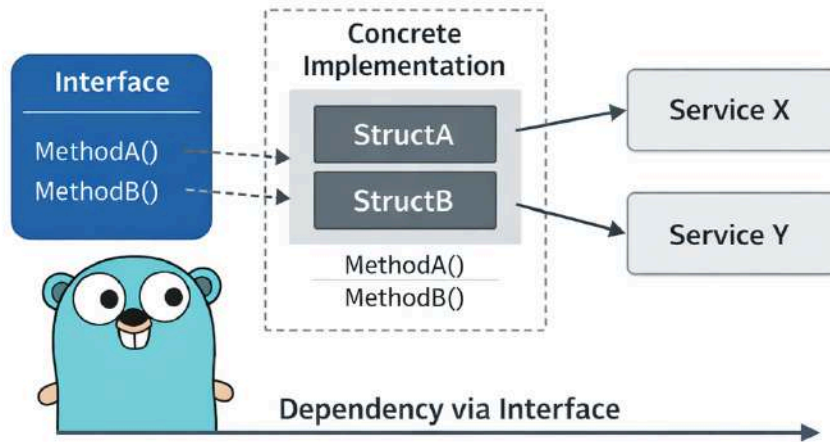
Relasi antara interface dan implementasi terbentuk secara implisit, namun konsistensinya tetap terjaga.



Abstraction di Backend

Abstraction menyembunyikan detail implementasi dan menampilkan perilaku penting, sehingga developer dapat fokus pada alur bisnis backend.

Hubungan Interface dan Abstraction



Interface di Go membantu menerapkan abstraction agar service bergantung pada kontrak, bukan implementasi konkret.



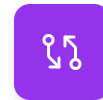
Dependensi pada Interface

Service menggunakan interface repository sehingga tidak terikat pada struct konkret.



Pisahkan Logika Bisnis

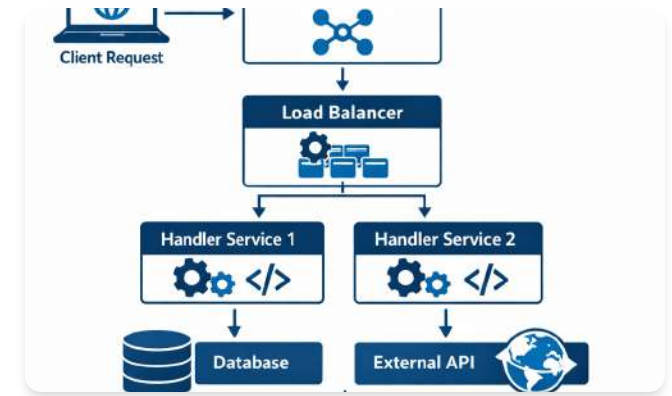
Abstraction menjaga logika bisnis tetap terpisah dari detail akses data.



Kode Lebih Mudah Dirawat

Struktur yang terpisah membuat kode lebih bersih, fleksibel, dan mudah dipelihara.

Alur Abstraction Backend



01 Handler ke Service

Handler menerima request lalu meneruskannya ke service agar logika aplikasi tetap terpisah.

03 Repository ke Database

Repository konkret menjalankan query ke database. Pola ini memudahkan testing dan mengurangi coupling.

02 Service pakai Interface

Service memanggil interface repository untuk mengambil data tanpa bergantung pada implementasi konkret.

Praktik Baik Map dan Interface



Pilih map untuk akses cepat

Gunakan map saat perlu mengambil data berdasarkan key dengan efisien.



Gunakan interface seperlunya

Terapkan interface pada batas antar layer, dan hindari abstraksi berlebihan.



Utamakan slice untuk urutan

Pilih slice bila urutan elemen dan iterasi yang konsisten lebih penting.



Kesalahan Umum yang Perlu Dihindari



Map tanpa cek key

Mengakses map tanpa memeriksa key dapat memicu bug dan perilaku yang tidak terduga.



Logika dan implementasi tercampur

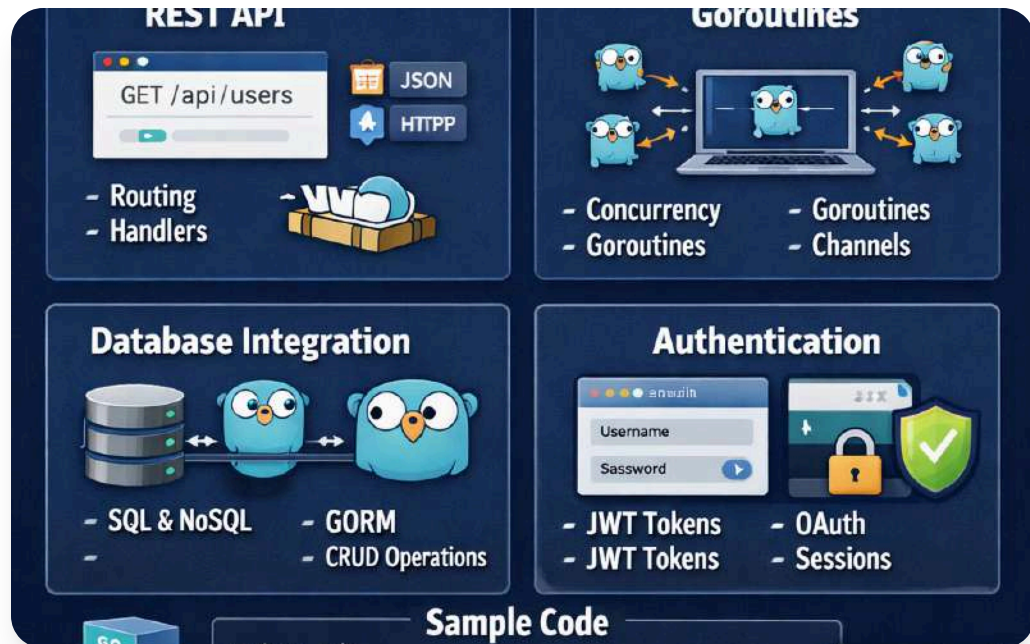
Mencampur logika bisnis dengan detail teknis membuat aplikasi membingungkan saat berkembang.



Interface terlalu besar

Interface yang terlalu luas membuat kode sulit dipahami, diuji, dan dipelihara.





Ringkasan Pertemuan

Dibahas *collection*, *map*, *interface*, dan *abstraction* untuk membangun aplikasi Go yang modular dan scalable.